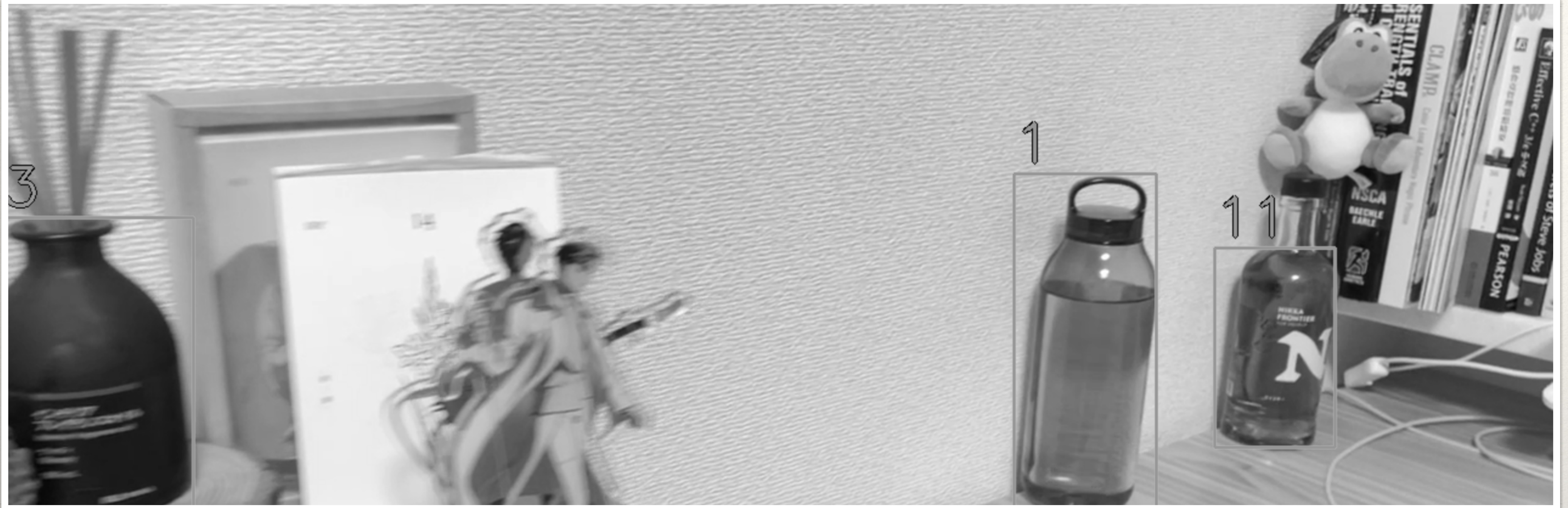




KunChun Tsai, 2024-10-20

Corpy&Co Assignment

Object Tracking C++ Development

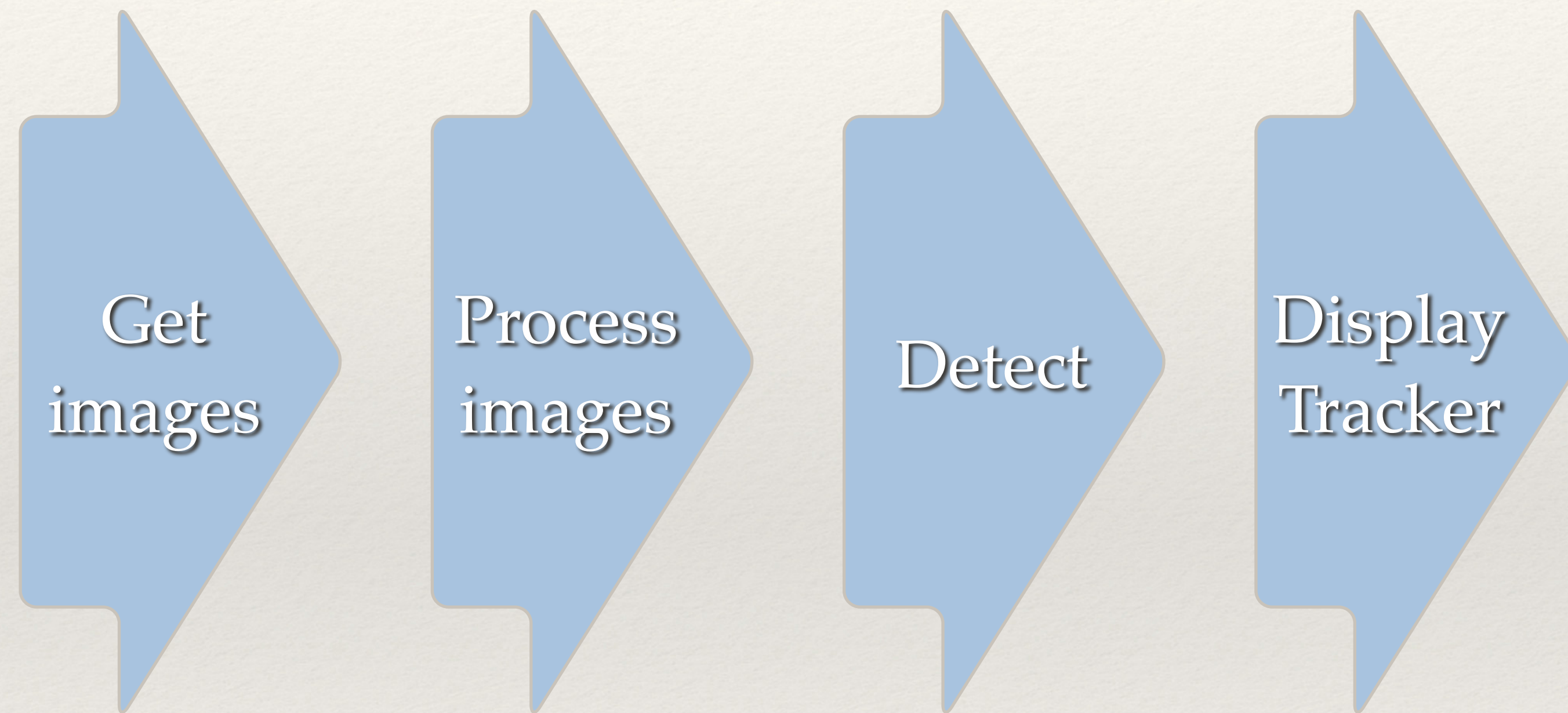
Introduction

- ❖ After walking through the assignment, I'd like to spend a little time to investigate object tracking models and dig into some of them to have a better concepts for the question 1
- ❖ My expertise is in C++ development and parallel optimization, so I'd like to obtain certain knowledge of models inferences and focus on more the C++ assignment in the question 2

Model Selection

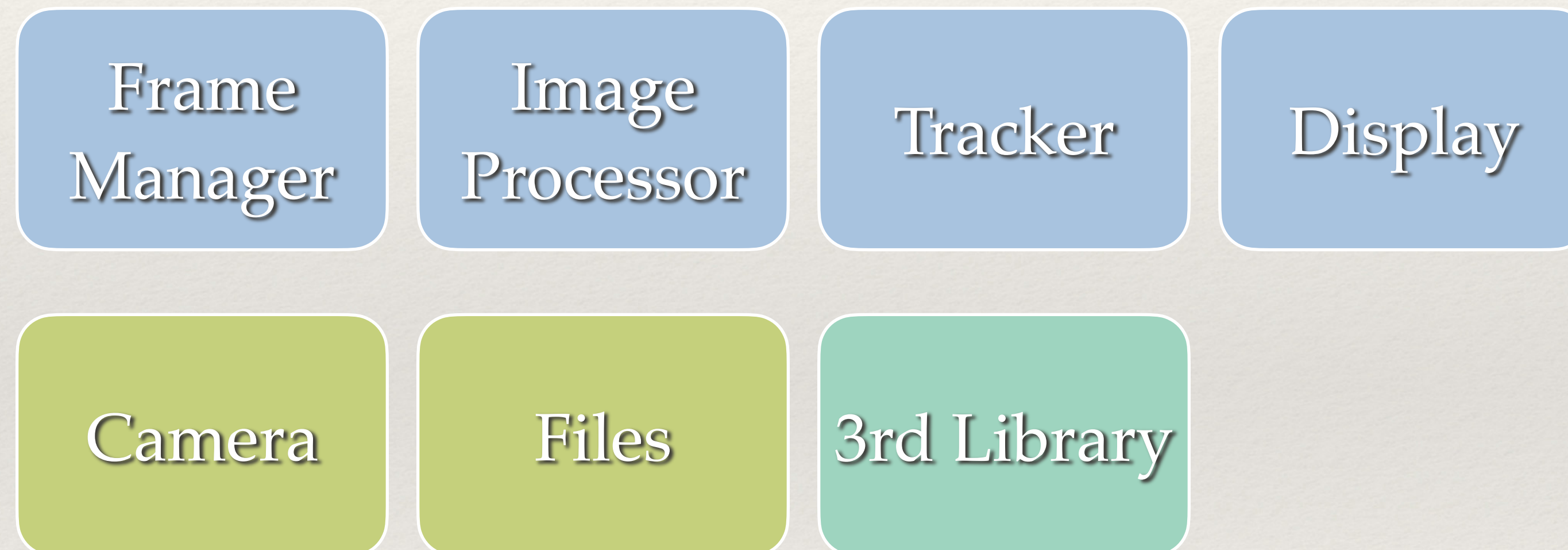
- ❖ I don't have much experience on model implementation, training and evaluations so I'll jump into object tracking model investigation and find an existing object tracking model for further development.
- ❖ Considering the use case is to track object from either a video file or camera stream, the real-time v.s. accuracy would be the factor that we select a model.
- ❖ Comparing with YOLO and Faster R-CNN, YOLO would be a good fit for this use case. In the sense that YOLO takes advantage of real-time object tracking versus Faster R-CNN with less frame rate but higher accuracy

Object Tracking Data Pipeline

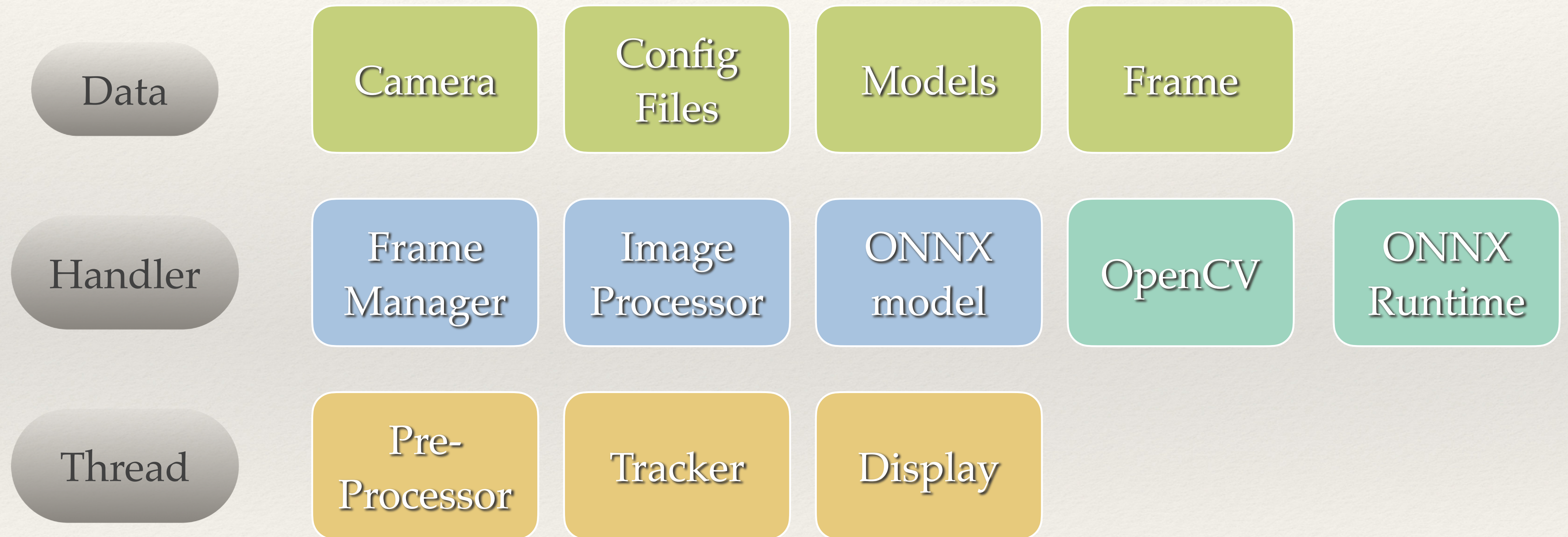


Object Tracking System

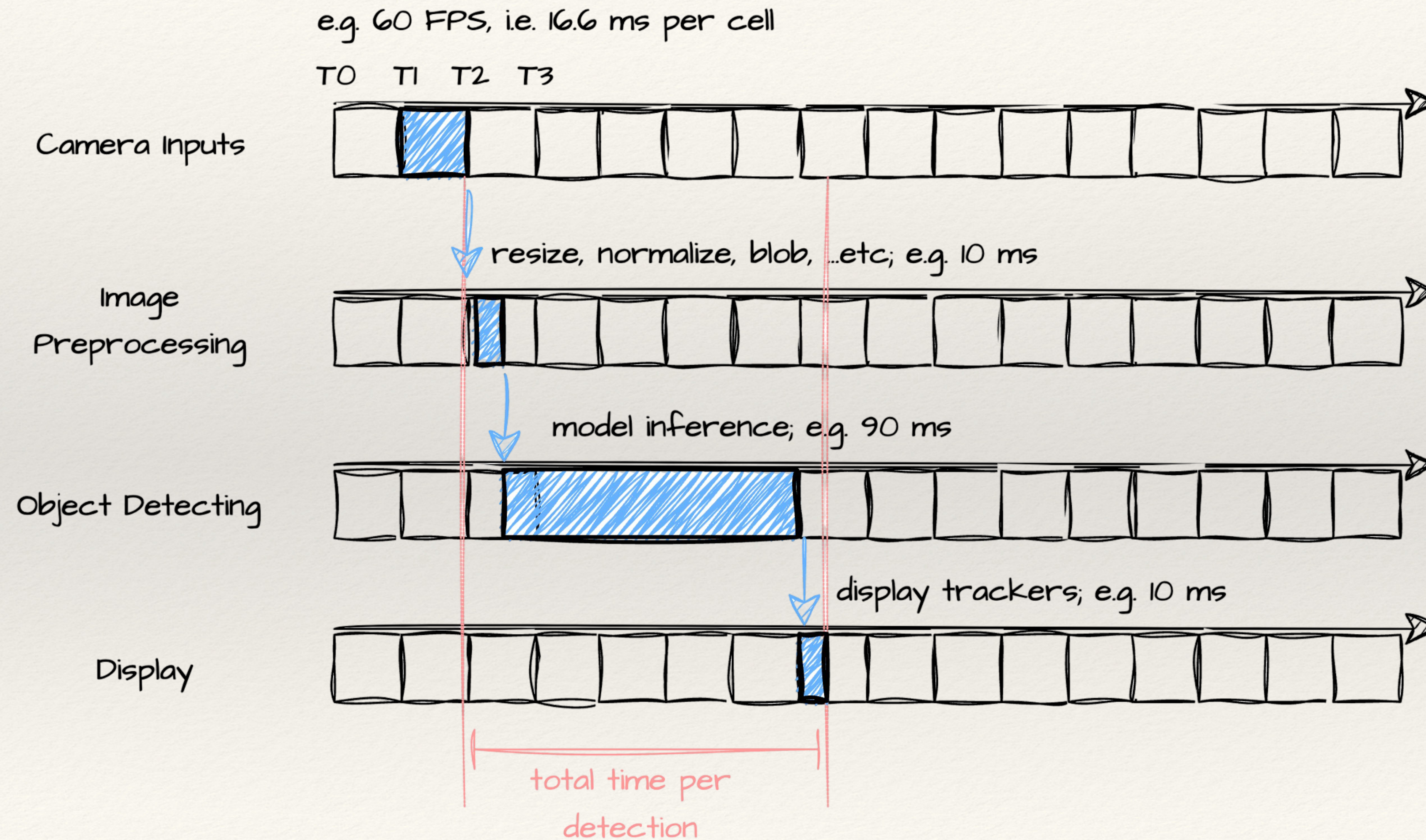
❖ Hight level modules for main functions



Software Building Blocks



Processing Speed Analysis



Threading

- ❖ According to the processing speed analysis, we understand that
 - ❖ Since the processing speeds are different among different processes, isolate them properly would be helpful for not blocking other processes and improve the performance.
- ❖ We have three threads in the program. (See the appendix for dependency graphs)
 - ❖ Main thread to handle input data and visualize trackers on frames
 - ❖ Preprocessor thread to dedicate on image processing and make images ready for detectors
 - ❖ Tracker thread is the primary process to run model inference and tracking.

Performance Results

- ❖ The object tracking program runs about 9 frames per second.
- ❖ Currently, it runs with 4 threads and enables all graph optimizations in ONNX Runtime.

```
// Enable multithreading
session_options.SetInterOpNumThreads(4);
session_options.SetIntraOpNumThreads(2);

// Enable graph optimizations
session_options.SetGraphOptimizationLevel(GraphOptimizationLevel::ORT_ENABLE_ALL);
```

-- single thread --

```
DEBUG: [Preproc] Finished preproc and pushed to output queue
DEBUG: [ONNXModel] Inference time: 162845 µs
DEBUG: [ONNXModel] Postprocessing time: 1 µs
DEBUG: [Tracker] ONNX detection time: 162.859 ms
DEBUG: [Tracker] Track update time: 0.001 ms
DEBUG: [Tracker] Total frame processing time: 162.866 ms
```

-- multi-thread: 4 --

```
INFO: Real-time FPS (17:30:38): 9.05
DEBUG: [Preproc] Finished preproc and pushed to output queue
DEBUG: [ONNXModel] Inference time: 92301 µs
DEBUG: [ONNXModel] Postprocessing time: 1 µs
DEBUG: [Tracker] ONNX detection time: 92.317 ms
DEBUG: [Tracker] Track update time: 0.001 ms
DEBUG: [Tracker] Frame processing time: 92.325 ms
```

```
DEBUG: [ONNXModel] Inference time: 87662 µs
DEBUG: [ONNXModel] Postprocessing time: 1 µs
DEBUG: [Tracker] ONNX detection time: 87.678 ms
DEBUG: [Tracker] Track update time: 0.001 ms
DEBUG: [Tracker] Frame processing time: 87.687 ms
```

```
INFO: Profiling results (501 frames):
INFO:   Main thread avg time: 16.01 ms
INFO:   Preprocessor avg time: 1.81 ms
INFO:   Tracker avg time: 85.15 ms
INFO:   Total avg time per frame: 102.97 ms
INFO:   Average FPS: 9.71
```

Performance Analysis

- ❖ From the profiling, we can find that the ONNX detection is the main contributor. Then the next high one is main thread, which handles data input and display tasks. These two parts will be the target to focus on in the performance tuning.
- ❖ Memory footprints: We didn't check if there was unnecessary memory copy across the pipeline, and this is something we should check.

Performance Analysis (generic)

- ❖ Multithreading: Since the bottleneck is in model detection at the moment, we cannot really find the benefit from multithreading. Still, it needs to be checked if it works properly without any blocking for the performance.
- ❖ SIMD optimization: Used ARM NEON intrinsics to optimize the post processing step, and potentially to optimize the image preprocessing as well.

Performance Analysis (model)

- ❖ Graph optimization: We had enabled ONNX Runtime graph optimization, but it could be trade-off between precision and speeds. We need more study to decide that.
- ❖ CoreML execution provider: It would be great if we can leverage the Neural Engine on an M2 chip
- ❖ Model selection: Consider using a smaller, faster model if accuracy requirements allow.

Challenges

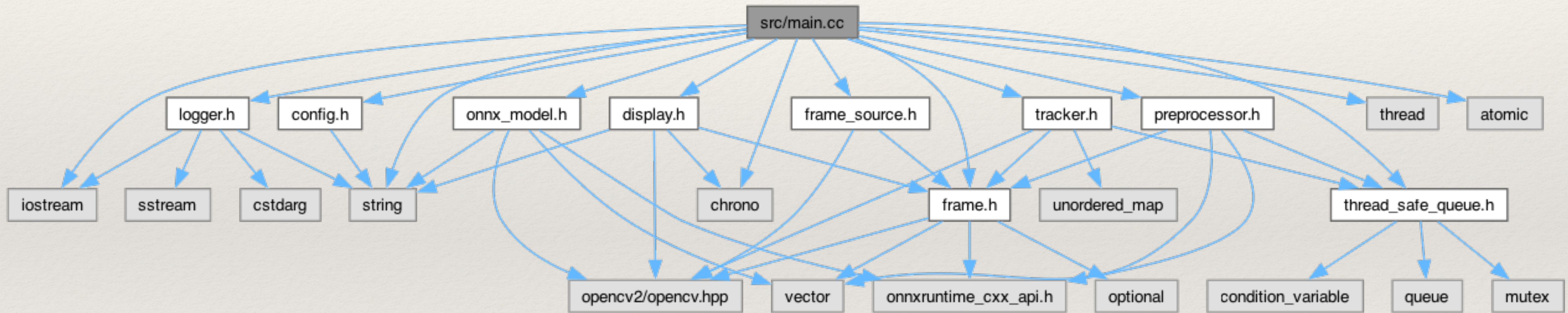
- ❖ The C++ development went smoothly for me. However, since the performance is highly dependent on the core algorithms, which is the model implementation, this is a big challenge to me and I need more time to dig deeper to understand it, then I can work on performance tuning.
- ❖ I spent most of my time on investigate how a model be loaded into a C++ project, parse the model and use the model
- ❖ The ONNX runtime with CoreML support took lots of time to setup as well, and it wasn't complete yet.

Improvement and Future Work

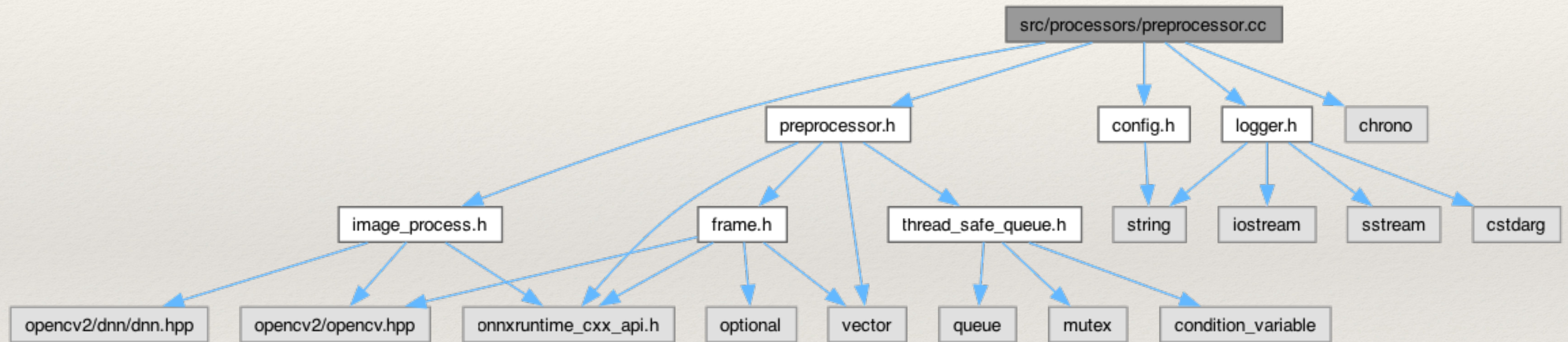
- ❖ Optimization
 - ❖ Setup the ONNX runtime with CoreML support
 - ❖ Review memory footprint and data alignment
 - ❖ SIMD optimization for image preprocessing and model post processing
 - ❖ Study models and optimize them
- ❖ Test
 - ❖ Complete unit test cases
 - ❖ Prepare test dataset

Appendix

Dependency: Main



Dependency: Preprocessor



Dependency: Tracker

